Handoff 34: Allowed imports and seeded random (master run WD2, ENG-07)

Date: 2026-11-02 (America/Toronto)
Written by: Claude Code (implementer), in a separate git worktree
For: the human maintainer, and ChatGPT or Claude chat (reviewer) without repository access
Status: **in_review**: one pull request (#39), not merged, not deployed. **No migration, no new dependency.** The worker's Content-Security-Policy is unchanged. **This is a security stage:** read section 3 first.

> **How to use this file:** upload it with `docs/plan/browser-checks.md` (section 9) and handoffs 11 and 29. The pull request diff is the source of truth (`build.md` section 14).

Starter prompt for the chat assistant

```
You are reviewing a security-related change to Python Academy: learner Python
runs in a Pyodide web worker in the learner's own browser, and this change adds
an import guard and seeded random. Read frontend/public/python-worker.js and
section 9 of docs/plan/browser-checks.md. Look for a blocked import that works by
accident, an allowed feature that breaks, and any claim that the guard is
stronger than it is. Python cannot sandbox itself: check that nothing relies on
it doing so.
```

Run plan details

Field	Value
Stage	WD2, ENG-07. The plan's first stage in Wave D with its gate met: WWD1 depends on S09, which is blocked
Base	`main`, at `8b5ef45`. No other PR needs to merge first
Part 2 boxes relied on	**None.** Its gate is "none; security stage"

1. Step 0

`git fetch --all`, `gh pr list --state all`: #32, #33 and #34 merged by the owner. #35, #36, #37 and #38 open. This PR is the fifth open, which is the cap, so the run stops after it. Nothing like this on any

branch. Worktree `../python-academy-imports`; the main folder was not switched.

2. Handoff record (build.md template)

Field	Value
Task	Allow only a listed set of standard-library modules, block the rest, and seed `random` from the harness
Human goal	Later units can use `math`, `random` and others with fair checks, and a learner does not wander into `os` or `js`
Non-goals	Any lesson; ENG-08 files; pinning or self-hosting Pyodide (SEC-05); a real sandbox
Data and compatibility	No migration. An optional `seed` on a case, sent only when set, so every existing plan is unchanged. **One changed message:** the `unsupported` feedback (section 4)
Deployment status	Not requested

Files changed

File	Why
`frontend/public/python-worker.js`	`ALLOWED_MODULES`, the guard, setup before it, and the seed. The worker's own later imports moved before the guard
`frontend/public/check-runner.js`	Passes a case's `seed` to the worker
`backend/app/content.py`, `backend/app/browser_checks.py`	The optional `seed`; the plan sends it when set; the corrected message
`backend/tests/case_runner.py`	Seeds `random` the way the worker does
`backend/tests/test_allowed_imports.py` (new, 17 tests), `frontend/tests/allowed-imports.spec.ts` (new, 19 tests), `frontend/tests/fixtures/seeded-random.json` (new)	Section 6
Fixtures `lesson-units-api.json`, `unit2-api.json`, `unit2-part2-api.json`, `unit3-api.json`	The corrected message
`frontend/tests/fixtures/pyodide-errors.json`	Internal line numbers of the worker's own code, 7 to 5 and 9 to 7. Learners never see these
`docs/plan/browser-checks.md`	Section 9, and two table rows
`docs/run-log.md`, this handoff and its PDF	The record

3. Security note

The plan says "Today any import is `unsupported`". That is not so. Run in real Pyodide on `main` at `8b5ef45`:

Module	Before this PR	After
`os`, `sys`, `subprocess`, `socket`, `urllib`, `shutil`, `pathlib`, `ctypes`, `importlib`, `builtins`, `pyodide`, `pyodide_js`, `js`	**Imported**	Refused
`math`, `random`, `datetime`, `json`, `csv`, `re`	Imported	Imported
`requests`, `turtle`, `pygame`, `micropip`	`ModuleNotFoundError`	The same

Was that a live security problem? I judged not, so I did not stop the run. Please check that judgement.

Question	Answer
Could learner code reach the network, through `js`?	`js.fetch`, `XMLHttpRequest`, `WebSocket` and `importScripts` were `None` (tested), and the worker's policy allows connections only to the Pyodide CDN
Could it reach the session?	A worker has no `document.cookie` and no page to read, and with no network it cannot send a request that would carry the cookie
Could it affect another user?	It runs only in that learner's own browser, on their own machine
Could it fake a result?	`js.postMessage` could send made-up reports. Someone can already do that with their browser's tools. `browser-checks.md` section 5 says this is practice-level evidence, not tamper-proof
What does `js` still expose?	`indexedDB` and `location`. Nothing in the app uses IndexedDB. The `sessionStorage` that `academy.tsx` uses is not available in a worker

What this PR changes, and what it does not. The guard makes accidents and curiosity hit a clear message. **It is not a sandbox and cannot be one:** `random._os.sys.modules` still reaches other modules, and a test records this. The boundary is the worker, as before: no network APIs, the same Content-Security-Policy. The plan's request to "check that no allowed module can import a blocked one" has this answer: **they do import them.** `random` imports `os`, and it has to. So this cannot be promised, and `browser-checks.md` says so.

4. How it works, and decisions

The guard replaces `__import__` after Pyodide has loaded and the network APIs are gone. An import from a module's own code goes through. Anything else, including `exec("import os")` and `__import__("os", {"__name__": "json"})`, must name a listed module, or it raises `ModuleNotFoundError: The module 'os' is not available here.` A check reports this as `unsupported`, with no change to the judge.

Decision	Options considered	Reason	Who decided
Six modules: `math`, `random`, `datetime`, `json`, `csv`, `re`	Add `collections`, `itertools`, `string`	"Exactly the modules the lessons need", from Part 8. The plan's "..." leaves room, and more can be added when a lesson needs one	Implementer (**assumption 1**)
Enforce in the worker	Parse imports on the server	A parser misses `exec`. The server never sees what runs, and Try it never reaches the server	Implementer
Try it is guarded too	Checks only	A program should not work on one card and be refused on the next	Implementer (**assumption 2**)
`sys` **is blocked, so** `import sys` **stops working**	Allow it	The plan lists it. This is the only visible change for a learner who already uses an import	Plan
Corrected the `unsupported` **message:** "These checks can use only a few of Python's modules, so that import can't be used here."	Keep "built-in features only"	The old text would be untrue, since `math` is not built in. This is the one change to an API answer, and only its wording	Implementer (**assumption 3**)
Default seed 0, and a `seed` per case	Require a seed	Any check is then repeatable without extra work	Implementer
Try it is not seeded	Seed it	A learner exploring `random` should see it vary	Implementer
`_strptime` is allowed as an internal name	Broader rule for names starting with `_`	C code imports it using the caller's globals, so `datetime.strptime` looked like the learner's import. A broad rule would allow `_socket`	Implementer
The list lives in the worker only	Also in the API	One list in one place, as the plan asks. The API doesn't need it	Plan

5. Bugs found and how they were fixed

Bug or near-miss	How it was found	Fix
The plan's claim about imports was wrong	Testing it in Pyodide before building	Section 3
`datetime.strptime` **would have broken**	Reasoning about how C code imports, then a test	`_strptime` allowed. Break B6 proves the test catches it

Bug or near-miss	How it was found	Fix
Break B5 got past my first tests: judging a dotted name by its last part failed nothing	Deliberate-break run	A new assertion that `import json.decoder` works. B5 now fails 2 tests
The recording of worker errors held internal line numbers	Full Playwright run	Updated; learners never see these numbers
A build failed with a symlinked `node_modules`	Probing	A real `npm ci` in this worktree. A link is enough for making a PDF, but not for a build
One full Playwright run failed many tests, cause not established	Full suite, first run: 194 passed, with failures across specs that use real Pyodide	I only kept the last lines of that log. It did not recur in 3 later full runs: 251 passed with the one real failure above, then 252 and 252. I suspect the Pyodide CDN, **but did not confirm it.** Watch for it in CI

6. How it was tested

Commands and results (each suite alone, macOS, Python 3.12.14, Node 24)

Command	Result
`cd backend && DATABASE_URL=sqlite:///<temp>.db ../../python-academy/backend/.venv/bin/python -m pytest -q`	**720 passed, 11 skipped**
Same with `TEST_POSTGRES_URL=<local PostgreSQL 16 from pgserver>`	**731 passed**
`cd frontend && npx tsc --noEmit && npm run build`	OK
`cd frontend && CI=1 npx playwright test`	**252 passed,** twice after a fresh build (see section 5)

What the tests cover

Test	Where
Every blocked module is refused, in both modes, in real Pyodide	`allowed-imports.spec.ts`
Every allowed module works, in both modes, including `datetime.strptime` and `import json.decoder`	Same
Other ways to import: `from`, dotted, `as`, `__import__`, with fake globals, `exec`, `eval`, a program setting `__name__`	Same
The way round the guard exists, and still reaches no network	Same
A refused import reports its type and the line; Try it shows a clear message	Same

Test	Where
Function-call cases still work	Same
Pyodide's seeded `random` equals CPython's, for a set seed and for none; each case restarts; the page's runner passes the seed; Try it varies	Same, and `test_allowed_imports.py`
The worker's policy is unchanged	Same
One list in one place; nothing blocked on it; the guard is on before learner code in both modes; the worker imports nothing after it; no lesson imports anything	`test_allowed_imports.py`
Every earlier suite, including `unit1-unchanged.spec.ts`	Unchanged and passing, apart from the two recordings in the table above

Mocks: the import tests talk to the real worker, with API calls answered 401 because no API is needed.
What this does not prove: behaviour on Safari or Firefox, or against someone determined to get round the guard, which it does not claim to stop.

Deliberate-break controls (each undone; files matched saved copies byte for byte afterwards)

What I broke	Tests that failed
B1. `os` is added to the list	**10**
B2. The guard is never switched on	**8**
B3. The guard is left out of Try it	**5**
B4. A program setting `__name__` counts as a module	**4**
B5. A dotted name is judged by its last part	**0 at first,** then **2** with the new assertion
B6. `datetime.strptime`'s internal import is not allowed	**2**
B7. The seed is ignored	**3**
B8. The runner drops the seed	**1**
B9. The old, untrue message returns	**5**

7. Known limitations

- **A learner can get round the guard on purpose.** That is by design, as section 3 says.
- `sys` **and others stop working for anyone already importing them.** No lesson does, and a check reports `unsupported`, so it never fails them wrongly.
- **Try it has no friendly translation for the refused import.** It shows Python's message, which names the module.

- **Each worker takes slightly longer to start,** because `random` loads every time. Not measured. The per-case 3 seconds starts after that.
- **Files:** `open()` still works on Pyodide's in-memory files. ENG-08 will cover this.

8. Live checks for you (after merging and deploying)

- [ ] In any Try it card, run `import os`: a clear message that `os` is not available here, and the page still works.
- [ ] Run `import math` and `print(math.floor(2.5))`: it prints 2.
- [ ] In a challenge, add `import os` at the top and run the check: "Not checked", with the new wording.
- [ ] Do a normal lesson from units 2 and 3 as usual: nothing has changed.
- [ ] **No CSP reports** in DevTools → Console.

9. Questions for you

1. **Section 3's judgement,** that the old imports were not a live security problem. *Recommended, not yet confirmed: agree, and still merge this soon.*
2. **Assumption 1, the six modules.** *Recommended, not yet confirmed: keep them, and add others only when a lesson needs one.*
3. **Assumption 2, guarding Try it.** *Recommended, not yet confirmed: keep.*
4. **Assumption 3, the new message.** *Recommended, not yet confirmed: keep.*
5. **Correct the plan's line "Today any import is `unsupported`"?** *Recommended, not yet confirmed: yes, in the next docs sync.*

10. For the docs sync

Where	What
Backlog ENG-07	proposed → in_review (#39, handoff 34)
Stage table	"34. ENG-07 | Allowed imports, seeded random | in_review (#39, handoff 34)"
`master-run.md` Part 9	Correct "Today any import is `unsupported`"
Decision log	A chapter from sections 3 to 6

11. Next concrete action

The run has stopped at the open-PR cap. Merge what you accept and answer the questions. Then, if you wish, tick boxes in Part 2 and run the same prompt.

12. Removing the worktree safely

From the **main** folder, after this PR is merged:

```
cd /Users/DEV/Projects/python-academy
git -C ../python-academy-imports status --short   # expect nothing, or only ??
frontend/AGENTS.md
rm -f ../python-academy-imports/frontend/AGENTS.md   # generated by Next.js, not part of
the project
git worktree remove ../python-academy-imports     # deletes only that folder; refuses if
anything is uncommitted
git worktree prune
git branch -d eng/allowlisted-imports             # refuses if the branch is not merged
```

Don't delete the folder in the Finder or with `rm -rf`; Git would keep a stale record of it (`git worktree prune` fixes that). If `git branch -d` refuses after a squash merge, check the PR says "Merged" on GitHub, then use `git branch -D eng/allowlisted-imports`.